

SQL

Vehicle Service & Maintenance System

Complete SQL Server Database Script

7 Tables

18 Procedures

5 Functions

4 Triggers

Sample Data

SQL Server 2019+ | Week 1 Project | 2026

SECTION 1 — TABLES (7 Tables)

Users Table — System Login & Access Control

Stores all Admin and Staff accounts. Role must be 'Admin' or 'Staff'.

Passwords are saved as hashed values. IsActive=0 means soft-deleted.

```
CREATE TABLE Users (
    UserID          INT IDENTITY(1,1) PRIMARY KEY,
    Username        VARCHAR(50)   NOT NULL UNIQUE,
    PasswordHash    VARCHAR(256)  NOT NULL,
    FullName        VARCHAR(100)  NOT NULL,
    Role            VARCHAR(20)   NOT NULL CHECK (Role IN ('Admin','Staff')),
    Email           VARCHAR(100)  UNIQUE,
    Phone           VARCHAR(20),
    IsActive        BIT NOT NULL  DEFAULT 1,
    CreatedAt       DATETIME NOT NULL DEFAULT GETDATE()
);
GO
```

Customers Table — Customer Management

Holds customer contact information. Each customer can own multiple vehicles.

Phone is required. Email must be unique across the system.

```
CREATE TABLE Customers (
    CustomerID      INT IDENTITY(1,1) PRIMARY KEY,
    FullName        VARCHAR(100)  NOT NULL,
    Email           VARCHAR(100)  UNIQUE,
    Phone           VARCHAR(20)   NOT NULL,
    Address         VARCHAR(255),
    CreatedAt       DATETIME NOT NULL DEFAULT GETDATE()
);
GO
```

Vehicles Table — Vehicle Management

Every vehicle belongs to one customer via CustomerID (FK).

LicensePlate and VIN must be unique. Year must be between 1900 and 2100.

```
CREATE TABLE Vehicles (
    VehicleID       INT IDENTITY(1,1) PRIMARY KEY,
    CustomerID      INT NOT NULL REFERENCES Customers(CustomerID),
    Make            VARCHAR(50)   NOT NULL,
    Model           VARCHAR(50)   NOT NULL,
    Year            INT NOT NULL CHECK (Year BETWEEN 1900 AND 2100),
    LicensePlate    VARCHAR(20)   NOT NULL UNIQUE,
    VIN             VARCHAR(50)   UNIQUE,
    Color           VARCHAR(30),
    Mileage         INT           CHECK (Mileage >= 0),
    CreatedAt       DATETIME NOT NULL DEFAULT GETDATE()
);
GO
```

ServiceSchedule Table — Booking / Scheduling

Records each service appointment. Links a vehicle and a customer to a date.

Status flow: Scheduled → In Progress → Completed / Cancelled.

```
CREATE TABLE ServiceSchedule (
    ScheduleID INT IDENTITY(1,1) PRIMARY KEY,
    VehicleID INT NOT NULL REFERENCES Vehicles(VehicleID),
    CustomerID INT NOT NULL REFERENCES Customers(CustomerID),
    ScheduledDate DATE NOT NULL,
    ServiceType VARCHAR(100) NOT NULL,
    Status VARCHAR(20) NOT NULL DEFAULT 'Scheduled'
        CHECK (Status IN ('Scheduled', 'In Progress', 'Completed', 'Cancelled')),
    Notes VARCHAR(500),
    CreatedBy INT NOT NULL REFERENCES Users(UserID),
    CreatedAt DATETIME NOT NULL DEFAULT GETDATE()
);
GO
```

ServiceDetails Table — Work Done & Cost

Records actual work performed. Linked to a schedule and a vehicle.

ServiceCost must be >= 0. CompletedAt is auto-set by a trigger.

```
CREATE TABLE ServiceDetails (
    ServiceID INT IDENTITY(1,1) PRIMARY KEY,
    ScheduleID INT NOT NULL REFERENCES ServiceSchedule(ScheduleID),
    VehicleID INT NOT NULL REFERENCES Vehicles(VehicleID),
    TechnicianID INT REFERENCES Users(UserID),
    ServiceDate DATE NOT NULL DEFAULT CAST(GETDATE() AS DATE),
    Description VARCHAR(1000) NOT NULL,
    Status VARCHAR(20) NOT NULL DEFAULT 'Pending'
        CHECK (Status IN ('Pending', 'In Progress', 'Completed')),
    ServiceCost DECIMAL(10,2) NOT NULL DEFAULT 0 CHECK (ServiceCost >= 0),
    PartsUsed VARCHAR(500),
    CompletedAt DATETIME,
    CreatedAt DATETIME NOT NULL DEFAULT GETDATE()
);
GO
```

Complaints Table — Complaint Management System

Customer complaints go here. Assigned to staff and tracked to resolution.

VehicleID is optional (complaint may not be about a specific vehicle).

```
CREATE TABLE Complaints (
    ComplaintID INT IDENTITY(1,1) PRIMARY KEY,
    CustomerID INT NOT NULL REFERENCES Customers(CustomerID),
    VehicleID INT REFERENCES Vehicles(VehicleID),
    Subject VARCHAR(200) NOT NULL,
    Description VARCHAR(2000) NOT NULL,
    Status VARCHAR(20) NOT NULL DEFAULT 'Open'
        CHECK (Status IN ('Open', 'In Progress', 'Resolved', 'Closed')),
    AssignedTo INT REFERENCES Users(UserID),
    ResolutionNotes VARCHAR(2000),
    CreatedAt DATETIME NOT NULL DEFAULT GETDATE(),
    ResolvedAt DATETIME
);
GO
```

AuditLog Table — Automatic Change Tracking

Populated entirely by triggers — never manually.

Records every INSERT, UPDATE, DELETE on critical tables with old and new values.

```
CREATE TABLE AuditLog (  
    LogID          INT IDENTITY(1,1) PRIMARY KEY,  
    TableName      VARCHAR(50) NOT NULL,  
    Action         VARCHAR(10) NOT NULL CHECK (Action IN ('INSERT','UPDATE','DELETE')),  
    RecordID       INT NOT NULL,  
    ChangedBy      VARCHAR(50),  
    ChangedAt      DATETIME NOT NULL DEFAULT GETDATE(),  
    OldValues      VARCHAR(MAX),  
    NewValues      VARCHAR(MAX)  
);  
GO
```

SECTION 2 — STORED PROCEDURES (18 Procedures)

USER Procedures

sp_AddUser — Create New User

Adds a new user. Checks for duplicate username before inserting.

```
CREATE OR ALTER PROCEDURE sp_AddUser
    @Username VARCHAR(50), @PasswordHash VARCHAR(256),
    @FullName VARCHAR(100), @Role VARCHAR(20),
    @Email VARCHAR(100) = NULL, @Phone VARCHAR(20) = NULL
AS BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM Users WHERE Username = @Username)
    BEGIN RAISERROR('Username already exists.',16,1); RETURN; END
    INSERT INTO Users (Username, PasswordHash, FullName, Role, Email, Phone)
    VALUES (@Username, @PasswordHash, @FullName, @Role, @Email, @Phone);
    SELECT SCOPE_IDENTITY() AS NewUserID;
END; GO
```

sp_UpdateUser / sp_DeleteUser

Updates user details. Delete is a soft-delete (sets IsActive=0).

```
CREATE OR ALTER PROCEDURE sp_UpdateUser
    @UserID INT, @FullName VARCHAR(100), @Role VARCHAR(20),
    @Email VARCHAR(100)=NULL, @Phone VARCHAR(20)=NULL, @IsActive BIT=1
AS BEGIN
    UPDATE Users SET FullName=@FullName, Role=@Role, Email=@Email,
    Phone=@Phone, IsActive=@IsActive WHERE UserID=@UserID;
END; GO

CREATE OR ALTER PROCEDURE sp_DeleteUser @UserID INT AS BEGIN
    UPDATE Users SET IsActive=0 WHERE UserID=@UserID;
END; GO
```

CUSTOMER Procedures

sp_AddCustomer / sp_UpdateCustomer / sp_DeleteCustomer

Add, update, or delete a customer. Delete is blocked if vehicles exist.

```
CREATE OR ALTER PROCEDURE sp_AddCustomer
    @FullName VARCHAR(100), @Email VARCHAR(100)=NULL,
    @Phone VARCHAR(20), @Address VARCHAR(255)=NULL
AS BEGIN
    INSERT INTO Customers (FullName, Email, Phone, Address)
    VALUES (@FullName, @Email, @Phone, @Address);
    SELECT SCOPE_IDENTITY() AS NewCustomerID;
END; GO

CREATE OR ALTER PROCEDURE sp_DeleteCustomer @CustomerID INT AS BEGIN
    IF EXISTS (SELECT 1 FROM Vehicles WHERE CustomerID=@CustomerID)
    BEGIN RAISERROR('Cannot delete customer with existing vehicles.',16,1); RETURN; END
    DELETE FROM Customers WHERE CustomerID=@CustomerID;
END; GO
```

VEHICLE Procedures

sp_AddVehicle / sp_UpdateVehicle / sp_DeleteVehicle

Register, update, or delete vehicles. Delete blocked if active schedule exists.

```

CREATE OR ALTER PROCEDURE sp_AddVehicle
    @CustomerID INT, @Make VARCHAR(50), @Model VARCHAR(50),
    @Year INT, @LicensePlate VARCHAR(20),
    @VIN VARCHAR(50)=NULL, @Color VARCHAR(30)=NULL, @Mileage INT=0
AS BEGIN
    INSERT INTO Vehicles (CustomerID,Make,Model,Year,LicensePlate,VIN,Color,Mileage)
    VALUES (@CustomerID,@Make,@Model,@Year,@LicensePlate,@VIN,@Color,@Mileage);
    SELECT SCOPE_IDENTITY() AS NewVehicleID;
END; GO

CREATE OR ALTER PROCEDURE sp_DeleteVehicle @VehicleID INT AS BEGIN
    IF EXISTS (SELECT 1 FROM ServiceSchedule
        WHERE VehicleID=@VehicleID AND Status IN ('Scheduled','In Progress'))
    BEGIN RAISERROR('Cannot delete vehicle with active service schedules.',16,1); RETURN; END
    DELETE FROM Vehicles WHERE VehicleID=@VehicleID;
END; GO

```

SERVICE SCHEDULE Procedures

sp_ScheduleService / sp_UpdateScheduleStatus

Book a vehicle for service. Update its status as work progresses.

```

CREATE OR ALTER PROCEDURE sp_ScheduleService
    @VehicleID INT, @CustomerID INT, @ScheduledDate DATE,
    @ServiceType VARCHAR(100), @Notes VARCHAR(500)=NULL, @CreatedBy INT
AS BEGIN
    INSERT INTO ServiceSchedule (VehicleID,CustomerID,ScheduledDate,ServiceType,Notes,CreatedBy)
    VALUES (@VehicleID,@CustomerID,@ScheduledDate,@ServiceType,@Notes,@CreatedBy);
    SELECT SCOPE_IDENTITY() AS NewScheduleID;
END; GO

CREATE OR ALTER PROCEDURE sp_UpdateScheduleStatus @ScheduleID INT, @Status VARCHAR(20) AS BEGIN
    UPDATE ServiceSchedule SET Status=@Status WHERE ScheduleID=@ScheduleID;
END; GO

```

SERVICE DETAILS Procedures

sp_AddServiceDetails / sp_UpdateServiceStatus

Record actual work done. Update status; completion time auto-set when Completed.

```

CREATE OR ALTER PROCEDURE sp_AddServiceDetails
    @ScheduleID INT, @VehicleID INT, @TechnicianID INT=NULL,
    @Description VARCHAR(1000), @ServiceCost DECIMAL(10,2)=0,
    @PartsUsed VARCHAR(500)=NULL
AS BEGIN
    INSERT INTO ServiceDetails
        (ScheduleID,VehicleID,TechnicianID,Description,ServiceCost,PartsUsed)
    VALUES (@ScheduleID,@VehicleID,@TechnicianID,@Description,@ServiceCost,@PartsUsed);
    SELECT SCOPE_IDENTITY() AS NewServiceID;
END; GO

CREATE OR ALTER PROCEDURE sp_UpdateServiceStatus
    @ServiceID INT, @Status VARCHAR(20),
    @ServiceCost DECIMAL(10,2)=NULL, @PartsUsed VARCHAR(500)=NULL
AS BEGIN
    UPDATE ServiceDetails SET Status=@Status,
        ServiceCost=ISNULL(@ServiceCost,ServiceCost),
        PartsUsed=ISNULL(@PartsUsed,PartsUsed),
        CompletedAt=CASE WHEN @Status='Completed' THEN GETDATE() ELSE CompletedAt END
    WHERE ServiceID=@ServiceID;
END; GO

```

COMPLAINT Procedures

sp_AddComplaint / sp_AssignComplaint / sp_ResolveComplaint

Log, assign, and resolve complaints. ResolvedAt is auto-set on resolution.

```

CREATE OR ALTER PROCEDURE sp_AddComplaint
    @CustomerID INT, @VehicleID INT=NULL,
    @Subject VARCHAR(200), @Description VARCHAR(2000)
AS BEGIN
    INSERT INTO Complaints (CustomerID,VehicleID,Subject,Description)
    VALUES (@CustomerID,@VehicleID,@Subject,@Description);
    SELECT SCOPE_IDENTITY() AS NewComplaintID;
END; GO

CREATE OR ALTER PROCEDURE sp_AssignComplaint @ComplaintID INT, @AssignedTo INT AS BEGIN
    UPDATE Complaints SET AssignedTo=@AssignedTo, Status='In Progress'
    WHERE ComplaintID=@ComplaintID;
END; GO

CREATE OR ALTER PROCEDURE sp_ResolveComplaint
    @ComplaintID INT, @ResolutionNotes VARCHAR(2000)
AS BEGIN
    UPDATE Complaints SET Status='Resolved',
        ResolutionNotes=@ResolutionNotes, ResolvedAt=GETDATE()
    WHERE ComplaintID=@ComplaintID;
END; GO

```

REPORT Procedures

sp_GetServiceReport / sp_GetCustomerHistory / sp_GetVehicleMaintenanceRecords

Generate reports for services, customers, and vehicle maintenance history.

```
CREATE OR ALTER PROCEDURE sp_GetServiceReport
    @StartDate DATE=NULL, @EndDate DATE=NULL
AS BEGIN
    SELECT sd.ServiceID, c.FullName AS Customer,
           v.Make+' '+v.Model+' ('+CAST(v.Year AS VARCHAR)+')' AS Vehicle,
           v.LicensePlate, sd.ServiceDate, sd.Description,
           sd.Status, sd.ServiceCost, sd.PartsUsed, u.FullName AS Technician
    FROM ServiceDetails sd
    JOIN Vehicles v    ON sd.VehicleID=v.VehicleID
    JOIN Customers c   ON v.CustomerID=c.CustomerID
    LEFT JOIN Users u  ON sd.TechnicianID=u.UserID
    WHERE (@StartDate IS NULL OR sd.ServiceDate>=@StartDate)
           AND (@EndDate IS NULL OR sd.ServiceDate<=@EndDate)
    ORDER BY sd.ServiceDate DESC;
END; GO
```

SECTION 3 — FUNCTIONS (5 Functions)

fn_GetVehicleTotalCost

Returns total cost of all completed services for a given vehicle.

```
CREATE OR ALTER FUNCTION fn_GetVehicleTotalCost (@VehicleID INT)
RETURNS DECIMAL(10,2) AS BEGIN
    DECLARE @Total DECIMAL(10,2);
    SELECT @Total = ISNULL(SUM(ServiceCost),0)
    FROM ServiceDetails
    WHERE VehicleID=@VehicleID AND Status='Completed';
    RETURN @Total;
END; GO
```

fn_GetCustomerServiceCount

Returns the total number of service records for a customer across all vehicles.

```
CREATE OR ALTER FUNCTION fn_GetCustomerServiceCount (@CustomerID INT)
RETURNS INT AS BEGIN
    DECLARE @Count INT;
    SELECT @Count = COUNT(*)
    FROM ServiceDetails sd
    JOIN Vehicles v ON sd.VehicleID = v.VehicleID
    WHERE v.CustomerID = @CustomerID;
    RETURN @Count;
END; GO
```

fn_HasActiveSchedule

Returns 1 if the vehicle has a current 'Scheduled' or 'In Progress' booking, else 0.

```
CREATE OR ALTER FUNCTION fn_HasActiveSchedule (@VehicleID INT)
RETURNS BIT AS BEGIN
    DECLARE @Result BIT = 0;
    IF EXISTS (SELECT 1 FROM ServiceSchedule
    WHERE VehicleID=@VehicleID AND Status IN ('Scheduled','In Progress'))
        SET @Result = 1;
    RETURN @Result;
END; GO
```

fn_GetVehicleLabel

Returns a human-readable vehicle label e.g. 'Toyota Corolla 2020 [RIW-1234]'.

```
CREATE OR ALTER FUNCTION fn_GetVehicleLabel (@VehicleID INT)
RETURNS VARCHAR(200) AS BEGIN
    DECLARE @Label VARCHAR(200);
    SELECT @Label = Make+' '+Model+' '+CAST(Year AS VARCHAR)+' ['+LicensePlate+']'
    FROM Vehicles WHERE VehicleID=@VehicleID;
    RETURN ISNULL(@Label,'Unknown Vehicle');
END; GO
```

fn_GetOpenComplaintCount

Returns the number of Open or In-Progress complaints for a customer.

```
CREATE OR ALTER FUNCTION fn_GetOpenComplaintCount (@CustomerID INT)
RETURNS INT AS BEGIN
    DECLARE @Count INT;
    SELECT @Count = COUNT(*) FROM Complaints
    WHERE CustomerID=@CustomerID AND Status IN ('Open','In Progress');
    RETURN @Count;
END; GO
```

Usage Examples

```
-- Total service cost for vehicle ID 1
SELECT dbo.fn_GetVehicleTotalCost(1) AS TotalCost;

-- How many services has customer 1 had?
SELECT dbo.fn_GetCustomerServiceCount(1) AS ServiceCount;

-- Is vehicle 3 currently booked?
SELECT dbo.fn_HasActiveSchedule(3) AS IsBooked;

-- Get readable vehicle label
SELECT dbo.fn_GetVehicleLabel(1) AS VehicleLabel;

-- How many open complaints for customer 1?
SELECT dbo.fn_GetOpenComplaintCount(1) AS OpenComplaints;
```

SECTION 4 — TRIGGERS (4 Triggers)

trg_ServiceDetails_Insert — AFTER INSERT on ServiceDetails

Fires automatically when service work is recorded.

Moves the linked schedule from 'Scheduled' to 'In Progress'. Also writes to AuditLog.

```
CREATE OR ALTER TRIGGER trg_ServiceDetails_Insert
ON ServiceDetails AFTER INSERT AS BEGIN
    SET NOCOUNT ON;
    -- Auto-update schedule to In Progress
    UPDATE ServiceSchedule SET Status='In Progress'
    FROM ServiceSchedule ss JOIN inserted i ON ss.ScheduleID=i.ScheduleID
    WHERE ss.Status='Scheduled';
    -- Write audit log
    INSERT INTO AuditLog (TableName,Action,RecordID,ChangedAt,NewValues)
    SELECT 'ServiceDetails','INSERT',i.ServiceID,GETDATE(),
        'ScheduleID='+CAST(i.ScheduleID AS VARCHAR)+
        ', VehicleID='+CAST(i.VehicleID AS VARCHAR)+
        ', Cost='+CAST(i.ServiceCost AS VARCHAR)
    FROM inserted i;
END; GO
```

trg_ServiceDetails_Update — AFTER UPDATE on ServiceDetails

Fires when service status is updated.

If status = 'Completed', the linked schedule is also marked 'Completed'. Logs the change.

```
CREATE OR ALTER TRIGGER trg_ServiceDetails_Update
ON ServiceDetails AFTER UPDATE AS BEGIN
    SET NOCOUNT ON;
    UPDATE ServiceSchedule SET Status='Completed'
    FROM ServiceSchedule ss JOIN inserted i ON ss.ScheduleID=i.ScheduleID
    WHERE i.Status='Completed';
    INSERT INTO AuditLog (TableName,Action,RecordID,ChangedAt,OldValues,NewValues)
    SELECT 'ServiceDetails','UPDATE',i.ServiceID,GETDATE(),
        'Status='+d.Status+', Cost='+CAST(d.ServiceCost AS VARCHAR),
        'Status='+i.Status+', Cost='+CAST(i.ServiceCost AS VARCHAR)
    FROM inserted i JOIN deleted d ON i.ServiceID=d.ServiceID;
END; GO
```

trg_Vehicles_Audit — AFTER INSERT/UPDATE/DELETE on Vehicles

Fires on any change to the Vehicles table.

Writes a full audit record showing what changed (plate, mileage, etc.).

```

CREATE OR ALTER TRIGGER trg_Vehicles_Audit
ON Vehicles AFTER INSERT, UPDATE, DELETE AS BEGIN
    SET NOCOUNT ON;
    IF EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1 FROM deleted)
        INSERT INTO AuditLog (TableName,Action,RecordID,ChangedAt,OldValues,NewValues)
        SELECT 'Vehicles','UPDATE',i.VehicleID,GETDATE(),
            'Plate='+d.LicensePlate+', Mileage='+CAST(ISNULL(d.Mileage,0) AS VARCHAR),
            'Plate='+i.LicensePlate+', Mileage='+CAST(ISNULL(i.Mileage,0) AS VARCHAR)
        FROM inserted i JOIN deleted d ON i.VehicleID=d.VehicleID;
    ELSE IF EXISTS(SELECT 1 FROM inserted)
        INSERT INTO AuditLog (TableName,Action,RecordID,ChangedAt,NewValues)
        SELECT 'Vehicles','INSERT',VehicleID,GETDATE(),
            'Make='+Make+', Model='+Model+', Plate='+LicensePlate FROM inserted;
    ELSE
        INSERT INTO AuditLog (TableName,Action,RecordID,ChangedAt,OldValues)
        SELECT 'Vehicles','DELETE',VehicleID,GETDATE(),
            'Make='+Make+', Model='+Model+', Plate='+LicensePlate FROM deleted;
END; GO

```

trg_Customers_Audit + trg_Complaints_Audit

Same audit pattern applied to Customers and Complaints tables.

Every create, change, or status update is recorded automatically.

```

-- Customers audit (INSERT / UPDATE / DELETE)
CREATE OR ALTER TRIGGER trg_Customers_Audit
ON Customers AFTER INSERT, UPDATE, DELETE AS BEGIN
    SET NOCOUNT ON;
    IF EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1 FROM deleted)
        INSERT INTO AuditLog (TableName,Action,RecordID,ChangedAt,OldValues,NewValues)
        SELECT 'Customers','UPDATE',i.CustomerID,GETDATE(),
            'Name='+d.FullName+', Phone='+d.Phone,
            'Name='+i.FullName+', Phone='+i.Phone
        FROM inserted i JOIN deleted d ON i.CustomerID=d.CustomerID;
END; GO

-- Complaints audit (INSERT / UPDATE)
CREATE OR ALTER TRIGGER trg_Complaints_Audit
ON Complaints AFTER INSERT, UPDATE AS BEGIN
    SET NOCOUNT ON;
    INSERT INTO AuditLog (TableName,Action,RecordID,ChangedAt,NewValues)
    SELECT 'Complaints','INSERT',ComplaintID,GETDATE(),
        'Subject='+Subject+', Status='+Status FROM inserted;
END; GO

```

SECTION 5 — SAMPLE DATA (For Testing)

Run these EXEC statements after creating all tables, procedures, functions, and triggers. They insert two users, two customers, two vehicles, one service booking, one service record, and one complaint — enough to test every feature.

```
-- Admin & Staff users
EXEC sp_AddUser 'admin', 'hashed_pw', 'System Admin', 'Admin', 'admin@vsms.com', '0300-0000001';
EXEC sp_AddUser 'ali_tech', 'hashed_pw', 'Ali Technician', 'Staff', 'ali@vsms.com', '0300-0000002';

-- Customers
EXEC sp_AddCustomer 'Ahmed Khan', 'ahmed@email.com', '0311-1111111', 'House 5, Rawalpindi';
EXEC sp_AddCustomer 'Sara Malik', 'sara@email.com', '0322-2222222', 'Flat 12, Islamabad';

-- Vehicles
EXEC sp_AddVehicle 1, 'Toyota', 'Corolla', 2020, 'RIW-1234', 'VIN001', 'White', 45000;
EXEC sp_AddVehicle 2, 'Honda', 'Civic', 2019, 'ISB-5678', 'VIN002', 'Black', 62000;

-- Schedule a service
EXEC sp_ScheduleService 1, 1, '2026-04-28', 'Oil Change', 'Customer requested synthetic oil', 1;

-- Record service details (triggers will auto-update schedule status)
EXEC sp_AddServiceDetails 1, 1, 2, 'Full oil change with filter replacement', 3500.00,
    'Oil Filter, 4L Synthetic Oil';

-- Log and assign a complaint
EXEC sp_AddComplaint 1, 1, 'Noise from engine', 'Strange knocking noise after last service.';
EXEC sp_AssignComplaint 1, 2;
GO
```

QUICK REFERENCE — Function & Report Calls

```
-- Total service cost for vehicle 1
SELECT dbo.fn_GetVehicleTotalCost(1) AS TotalCost;

-- Service count for customer 1
SELECT dbo.fn_GetCustomerServiceCount(1) AS ServiceCount;

-- Active schedule check
SELECT dbo.fn_HasActiveSchedule(1) AS IsActive;

-- Vehicle label
SELECT dbo.fn_GetVehicleLabel(1) AS VehicleLabel;

-- Open complaints
SELECT dbo.fn_GetOpenComplaintCount(1) AS OpenComplaints;

-- Reports
EXEC sp_GetServiceReport '2026-01-01','2026-12-31';
EXEC sp_GetCustomerHistory 1;
EXEC sp_GetVehicleMaintenanceRecords 1;
```